



(19) **United States**

(12) **Patent Application Publication**

(10) **Pub. No.: US 2025/0284640 A1**

(43) **Pub. Date: Sep. 11, 2025**

(71) **Wasef et al.**

(54) **SYSTEMS AND METHODS FOR GROUPING MEMORY ADDRESSES**

Publication Classification

(71) Applicant: **Samsung Electronics Co., Ltd.**,
Suwon-si (KR)

(72) Inventors: **Michael Wasef**, San Jose, CA (US);
Andrew Chang, Los Altos, CA (US);
Dongyang Li, Milpitas, CA (US);
Mohammadreza Soltaniyeh,
Sunnyvale, CA (US); **Luis Cargnini**,
San Jose, CA (US)

(51) **Int. Cl.**
G06F 12/0862 (2016.01)
G06F 12/02 (2006.01)
G06F 13/16 (2006.01)

(52) **U.S. Cl.**
CPC **G06F 12/0862** (2013.01); **G06F 12/0246**
(2013.01); **G06F 13/1657** (2013.01)

(21) Appl. No.: **18/772,031**

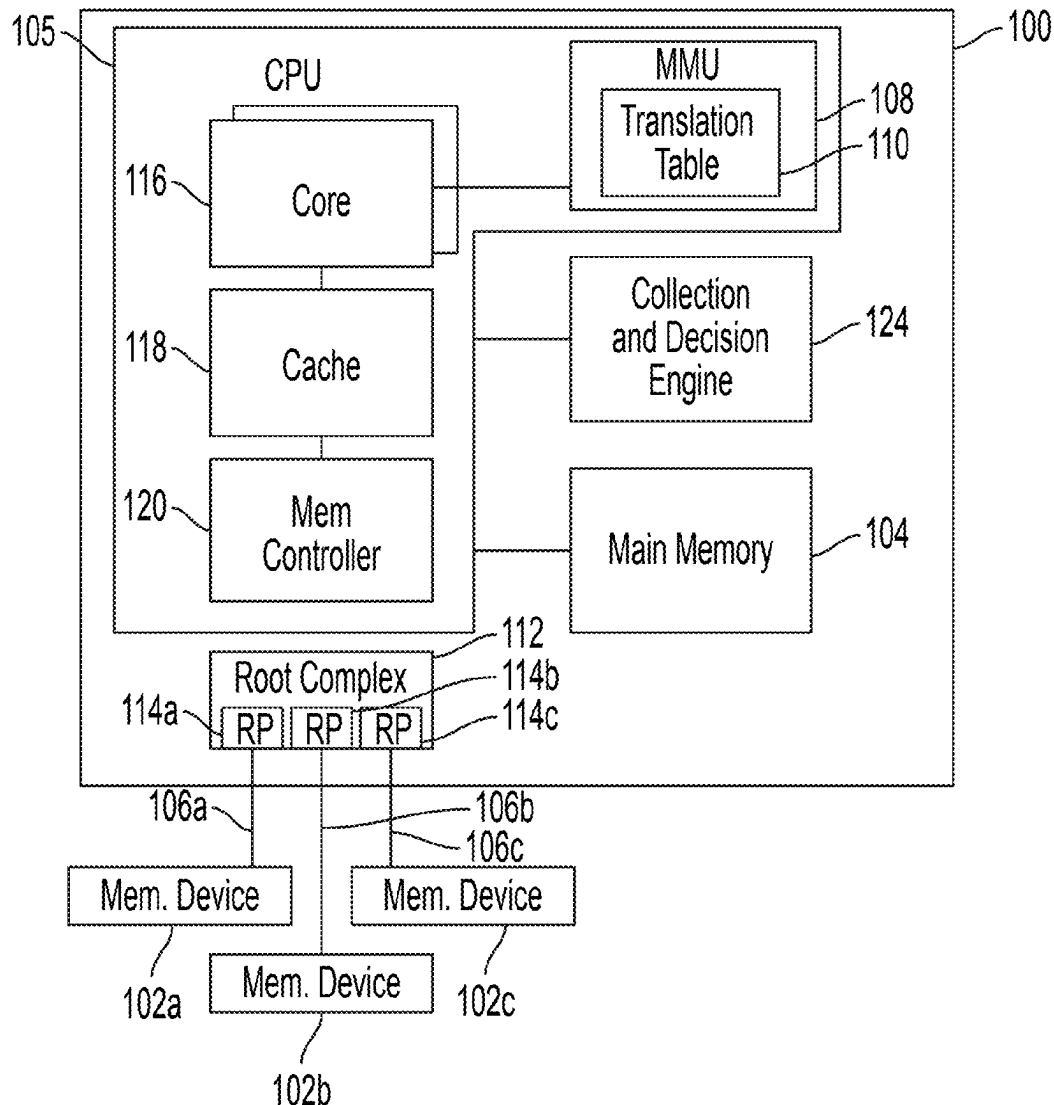
(22) Filed: **Jul. 12, 2024**

Related U.S. Application Data

(60) Provisional application No. 63/561,438, filed on Mar. 5, 2024.

(57) **ABSTRACT**

Systems and methods for grouping memory addresses are disclosed. A storage device may include a first storage medium, a second storage medium, and a processor configured to: identify a first memory address; identify a first region of the first storage medium based on the first memory address; track activity of the first region; identify a second memory address based on tracking activity of the first region; and retrieve data associated with the second memory address from the first storage medium and store the data in the second storage medium.



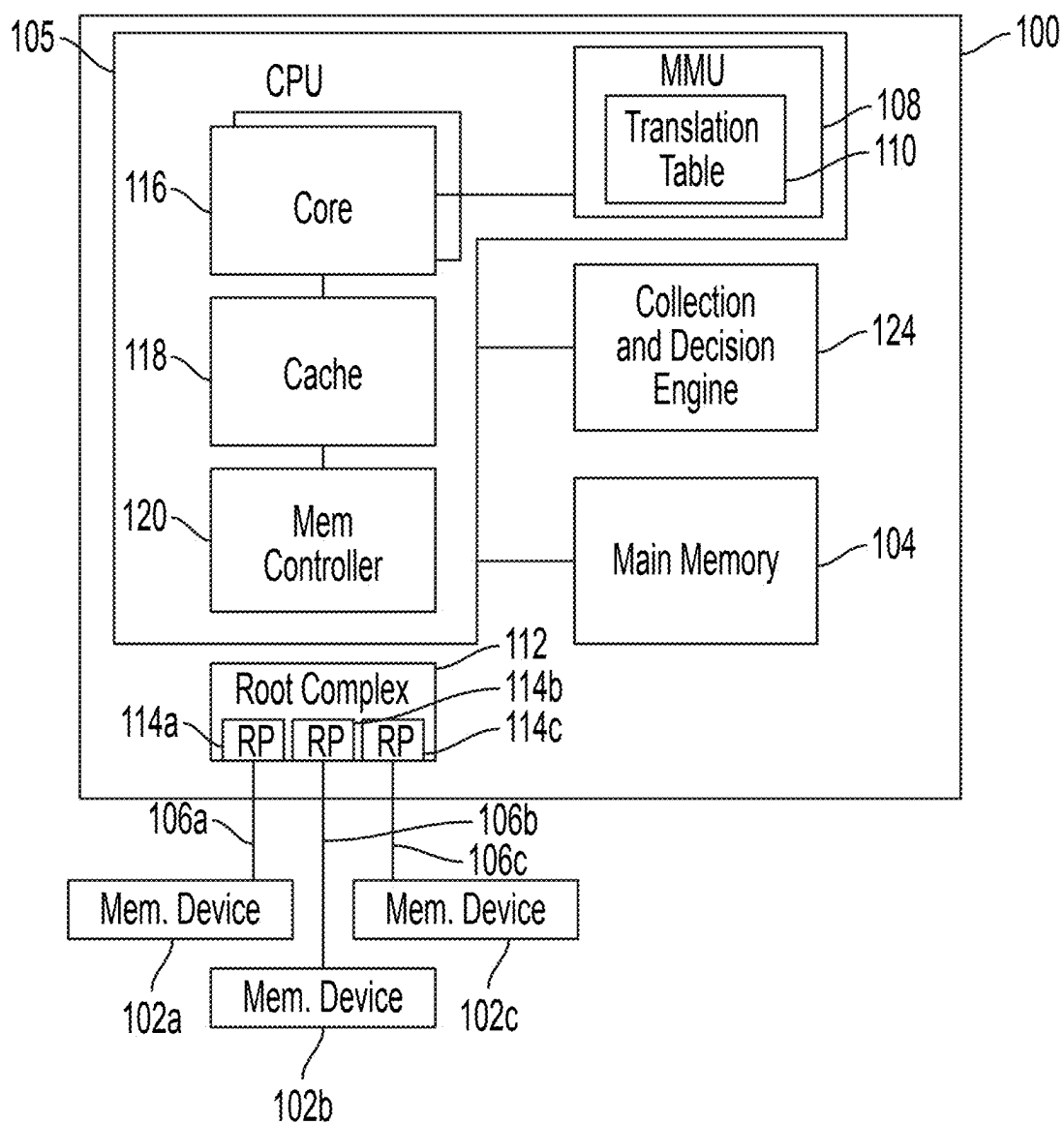


FIG. 1

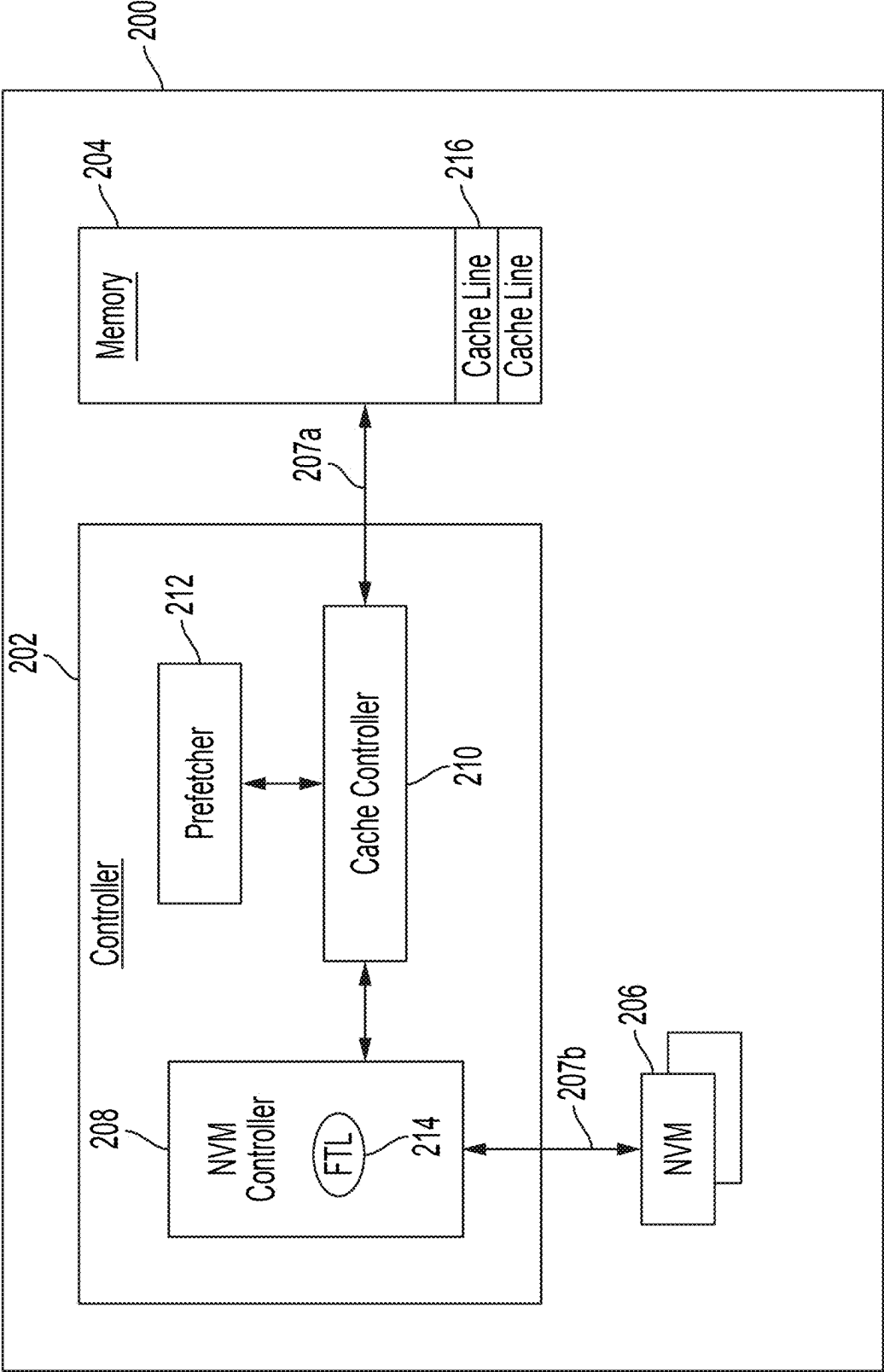


FIG. 2

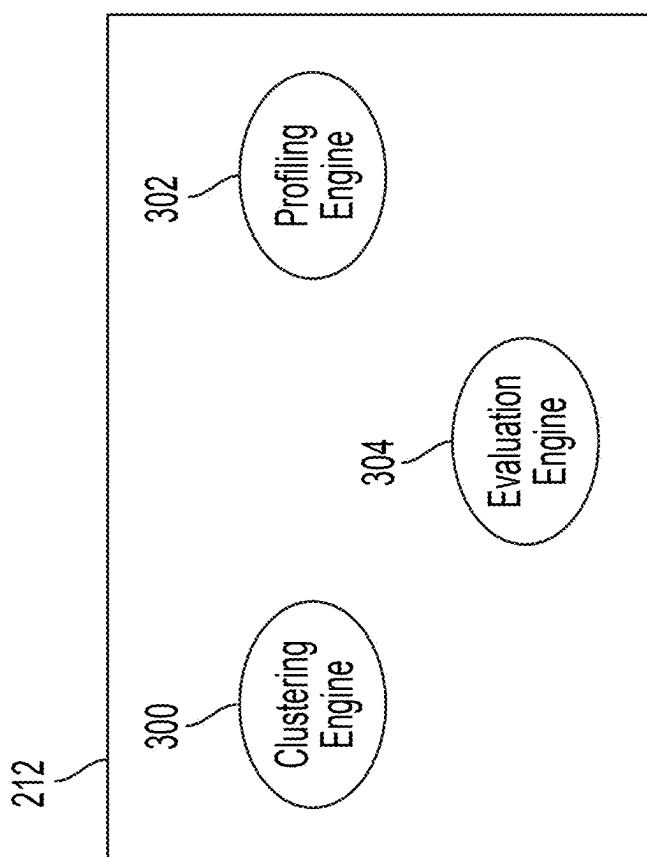
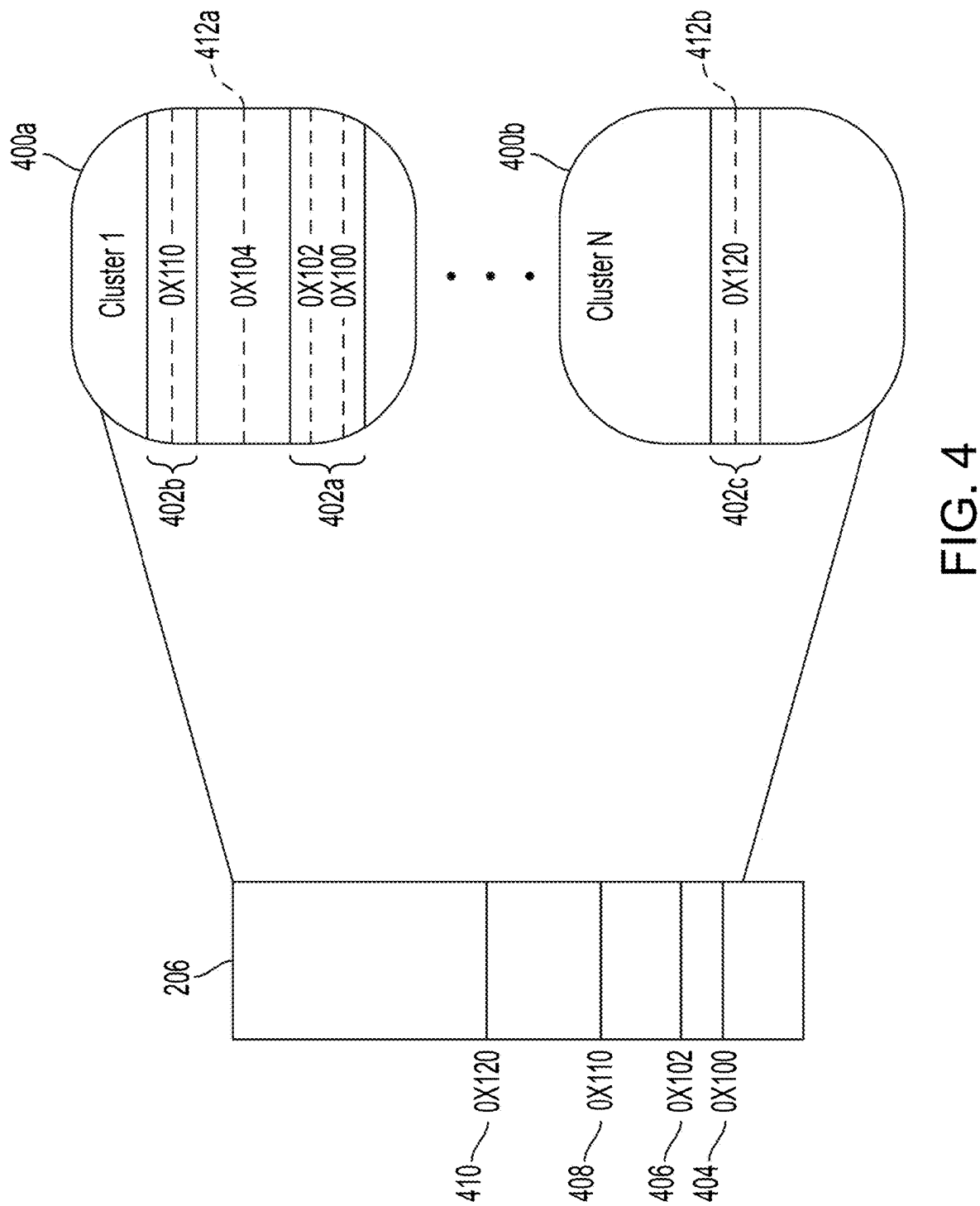


FIG. 3



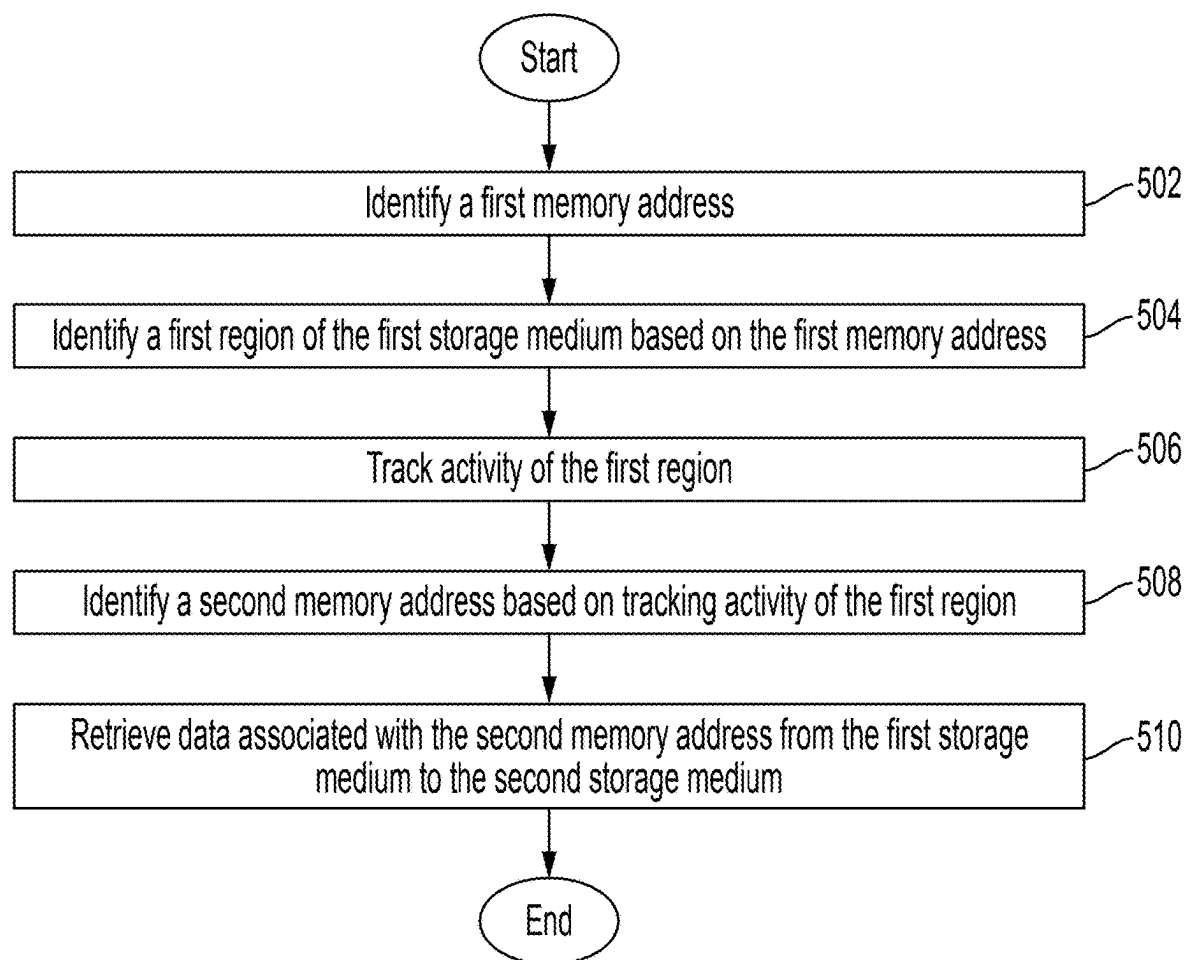


FIG. 5

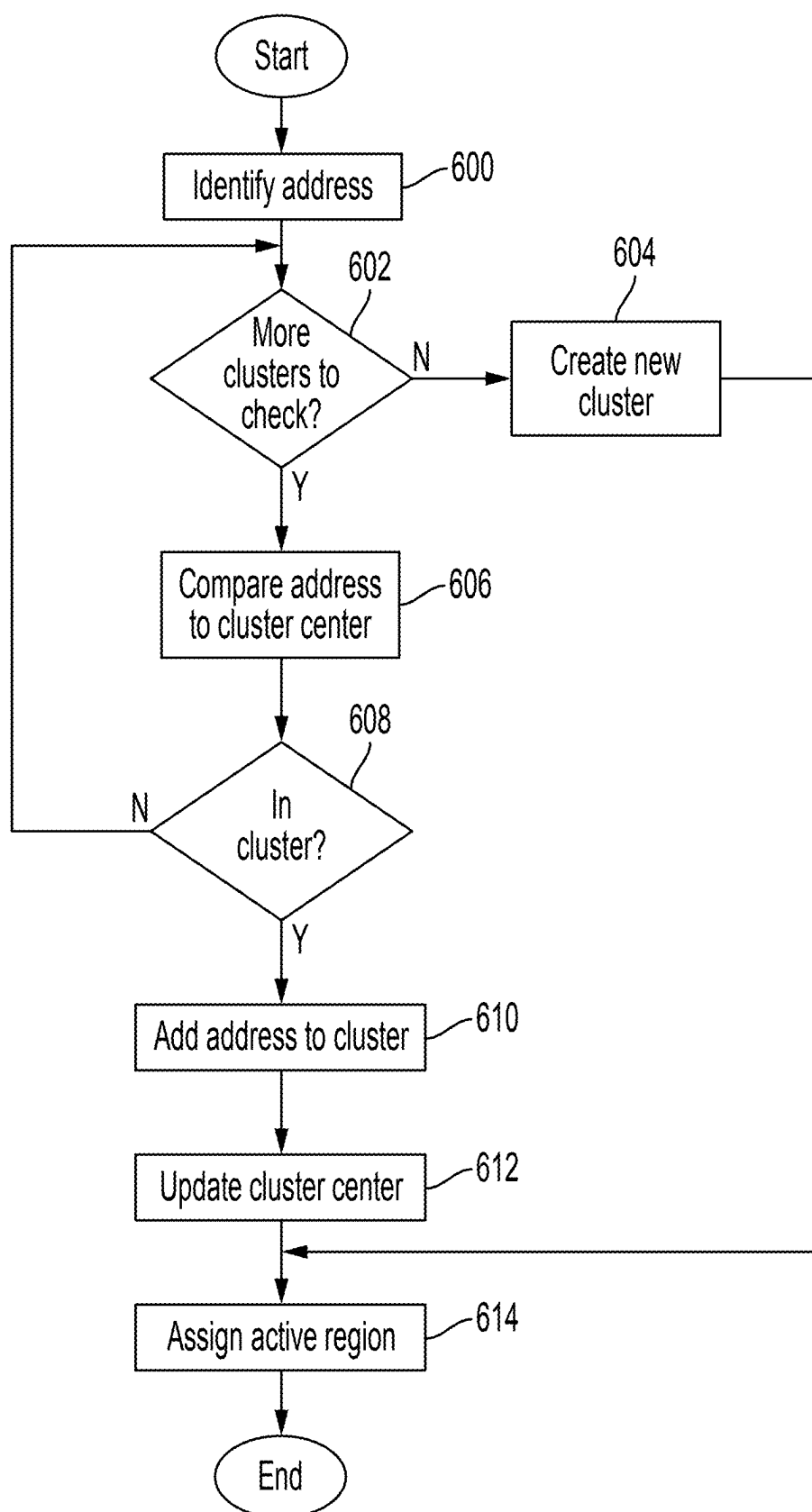


FIG. 6

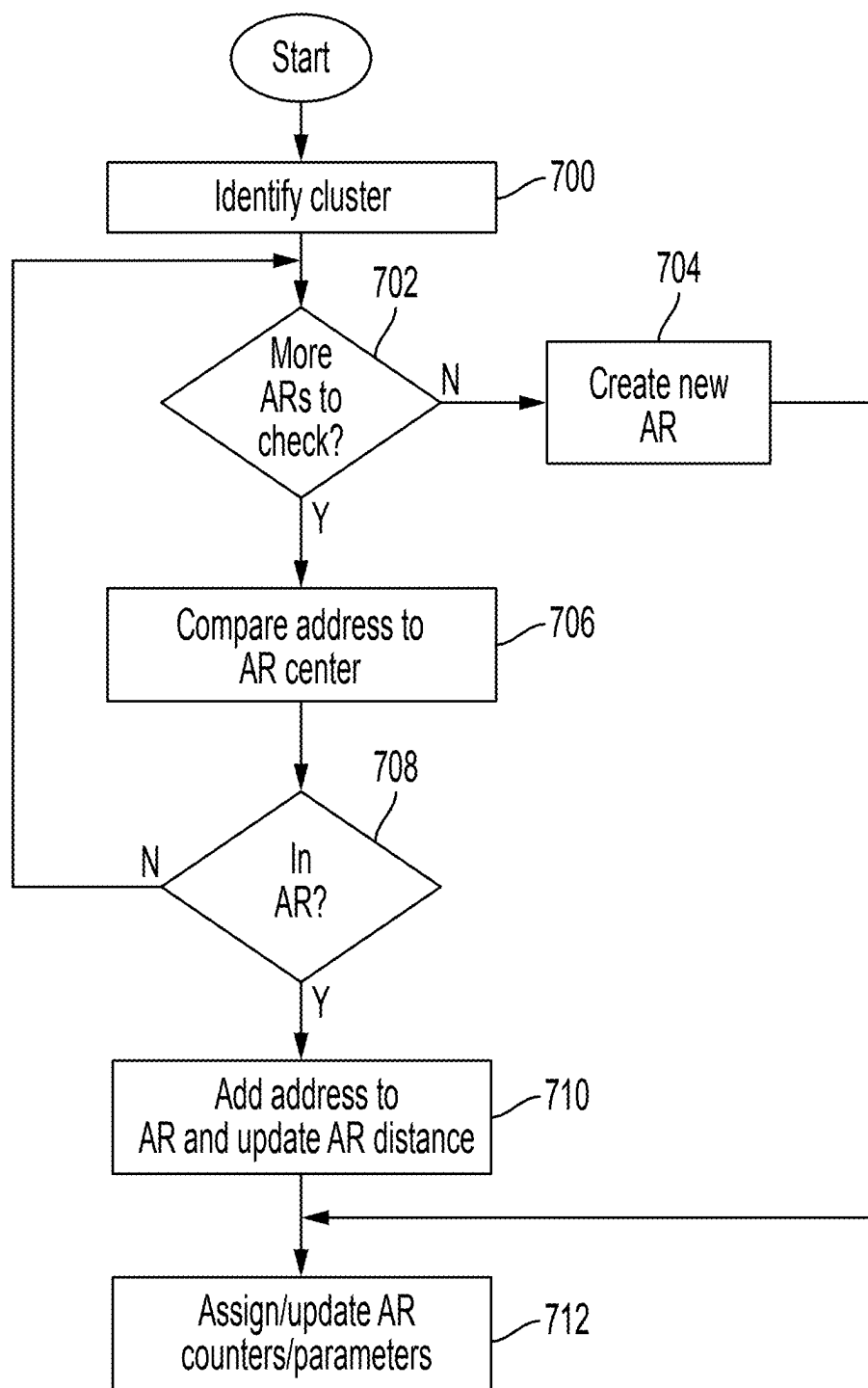


FIG. 7

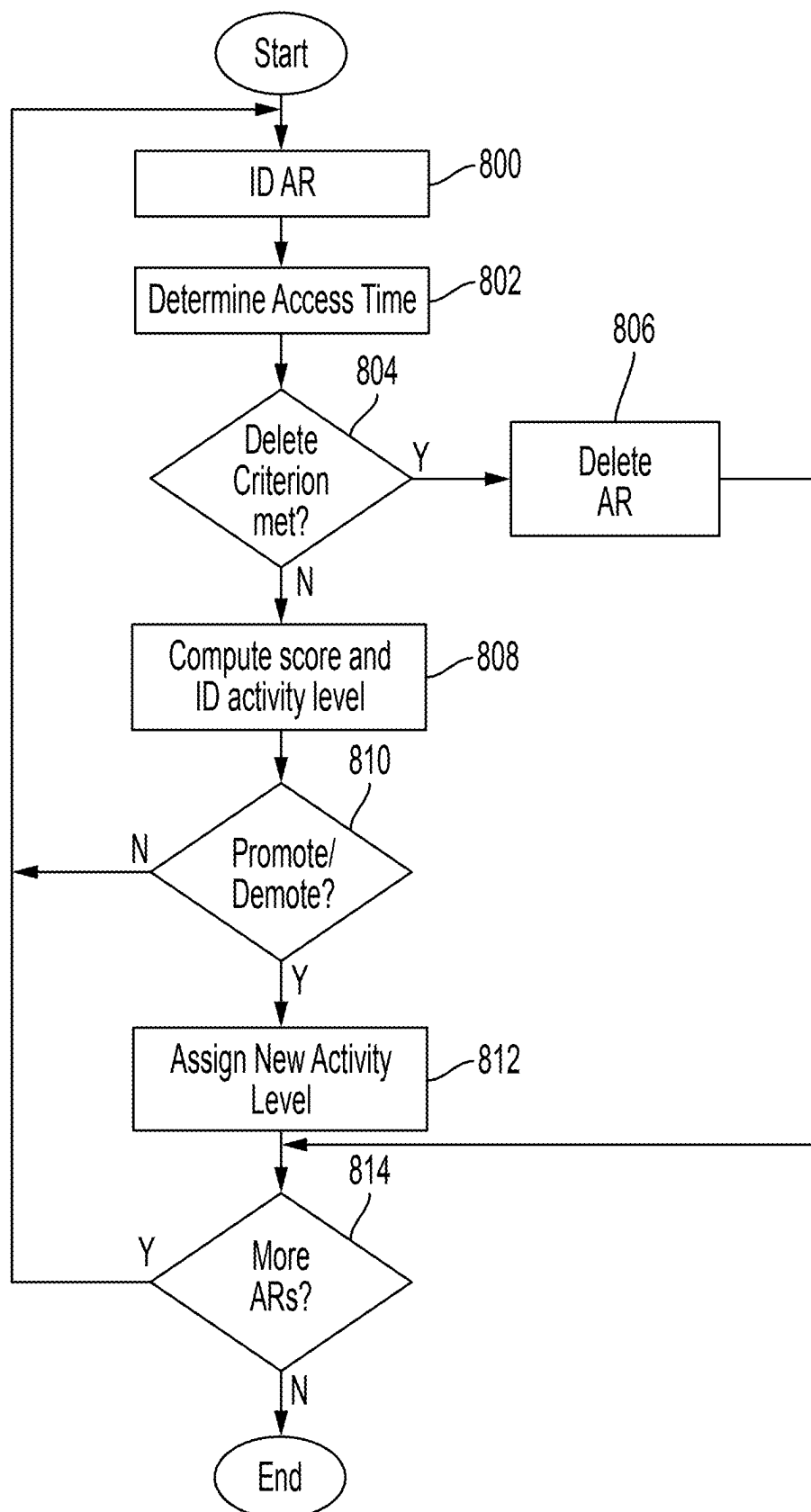


FIG. 8

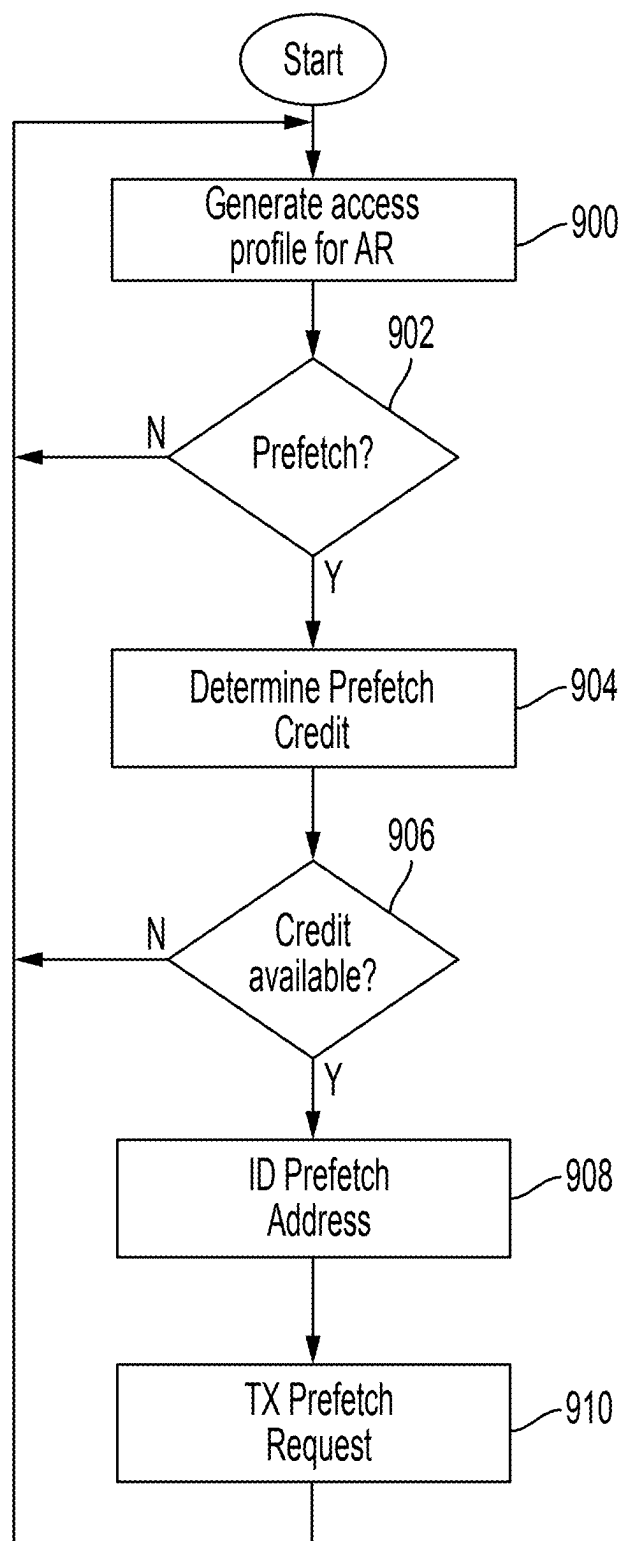


FIG. 9

SYSTEMS AND METHODS FOR GROUPING MEMORY ADDRESSES

CROSS-REFERENCE TO RELATED APPLICATION(S)

[0001] The present application claims priority to and the benefit of U.S. Provisional Application No. 63/561,438, filed Mar. 5, 2024, entitled “RUN-TIME ADAPTIVE CLUSTERING ALGORITHM TO PREDICT HOST ACCESS FOR EFFECTIVE PREFETCHING,” the entire content of which is incorporated herein by reference. The present application is also related to U.S. Application entitled “SYSTEMS AND METHODS FOR PATTERN RECOGNITION”, and U.S. Application entitled “SYSTEMS AND METHODS FOR IDENTIFYING REGIONS OF A MEMORY DEVICE”, both of which are filed on even date herewith, the content of both of which are incorporated herein by reference.

FIELD

[0002] One or more aspects of embodiments according to the present disclosure relate to memory devices, and more particularly to systems and methods for grouping memory addresses.

BACKGROUND

[0003] An application running on a host computing device may need to read and write data to memory. As the amount data read and written to memory increases, the demand for storage devices and memory, and efficiently accessing the storage devices and memory, may also increase.

[0004] The above information disclosed in this Background section is only for enhancement of understanding of the background of the present disclosure, and therefore, it may contain information that does not form prior art.

SUMMARY

[0005] One or more embodiments of the present disclosure are directed to a storage device comprising: a first storage medium; a second storage medium; and a processor configured to: identify a first memory address; identify a first region of the first storage medium based on the first memory address; track activity of the first region; identify a second memory address based on tracking activity of the first region; and retrieve data associated with the second memory address from the first storage medium and store the data in the second storage medium.

[0006] According to some embodiments, the first storage medium includes non-volatile memory and the second storage medium includes volatile memory.

[0007] According to some embodiments, the first region is associated with a first range of addresses, and the processor is further configured to: identify a third memory address from the first range of addresses; compute a difference between the first memory address and the third memory address; and determine that the difference satisfies a threshold value.

[0008] According to some embodiments the processor is further configured to: identify a second region based on the first memory address, wherein the second region is associated with a second range of addresses, wherein the second region is of a preset size.

[0009] According to some embodiments, the second region includes the first region, wherein the processor is configured to identify the first region includes the processor being configured to: identify the second region based on the first memory address; and identify the first region based on the processor being configured to identify the second region.

[0010] According to some embodiments, the processor is further configured to: update the third memory address based on the first memory address and the first range of memory addresses associated with the first region.

[0011] According to some embodiments, the processor is further configured to: assign a first value to the first region based on the activity of the first region; determine, based on the first value, a first number of accesses to the second storage medium allotted to the first region.

[0012] According to some embodiments, the processor is further configured to: identify an increase in the activity of the first region; assign a second value to the first region based on identifying the increase; and determine, based on the second value, a second number of accesses to the second storage medium allotted to the first region.

[0013] According to some embodiments, the processor is further configured to: divide the first region into a second region and a third region, wherein the second region is allotted the first number of accesses to the second storage medium, and the third region is allotted the first number of accesses to the second storage medium.

[0014] According to some embodiments, the processor is further configured to: identify a decrease in the activity of the first region; and change a status of the first region based on the decrease in the activity.

[0015] One or more embodiments of the present disclosure are also directed to a method comprising: identifying a first memory address; identifying a first region of a first storage medium based on the first memory address; tracking activity of the first region; identifying a second memory address based on the tracking of the activity of the first region; and retrieving data associated with the second memory address from the first storage medium and storing the data in a second storage medium.

[0016] These and other features, aspects and advantages of the embodiments of the present disclosure will be more fully understood when considered with respect to the following detailed description, appended claims, and accompanying drawings. Of course, the actual scope of the invention is defined by the appended claims.

BRIEF DESCRIPTION OF THE DRAWINGS

[0017] Non-limiting and non-exhaustive embodiments of the present embodiments are described with reference to the following figures, wherein like reference numerals refer to like parts throughout the various views unless otherwise specified.

[0018] FIG. 1 depicts a block diagram of a system for adaptive clustering according to one or more embodiments;

[0019] FIG. 2 depicts a block diagram of a memory device according to one or more embodiments;

[0020] FIG. 3 depicts a block diagram of a prefetch engine according to one or more embodiments;

[0021] FIG. 4 depicts a conceptual block diagram of clusters and active regions according to one or more embodiments;

[0022] FIG. 5 depicts a flow diagram of a process for adaptive clustering for data prefetching according to one or more embodiments;

[0023] FIG. 6 depicts a flow diagram of a process for assigning an address to a cluster according to one or more embodiments;

[0024] FIG. 7 depicts a block diagram of a process for assigning an address to an active region according to one or more embodiments;

[0025] FIG. 8 depicts a flow diagram of a process for evaluating active regions according to one or more embodiments; and

[0026] FIG. 9 depicts a flow diagram of a prefetching process according to one or more embodiments.

DETAILED DESCRIPTION

[0027] Hereinafter, example embodiments will be described in more detail with reference to the accompanying drawings, in which like reference numbers refer to like elements throughout. The present disclosure, however, may be embodied in various different forms, and should not be construed as being limited to only the illustrated embodiments herein. Rather, these embodiments are provided as examples so that this disclosure will be thorough and complete, and will fully convey the aspects and features of the present disclosure to those skilled in the art. Accordingly, processes, elements, and techniques that are not necessary to those having ordinary skill in the art for a complete understanding of the aspects and features of the present disclosure may not be described. Unless otherwise noted, like reference numerals denote like elements throughout the attached drawings and the written description, and thus, descriptions thereof may not be repeated. Further, in the drawings, the relative sizes of elements, layers, and regions may be exaggerated and/or simplified for clarity.

[0028] Embodiments of the present disclosure are described below with reference to block diagrams and flow diagrams. Thus, it should be understood that each block of the block diagrams and flow diagrams may be implemented in the form of a computer program product, an entirely hardware embodiment, a combination of hardware and computer program products, and/or apparatus, systems, computing devices, computing entities, and/or the like carrying out instructions, operations, steps, and similar words used interchangeably (for example the executable instructions, instructions for execution, program code, and/or the like) on a computer-readable storage medium for execution. For example, retrieval, loading, and execution of code may be performed sequentially such that one instruction is retrieved, loaded, and executed at a time. In some example embodiments, retrieval, loading, and/or execution may be performed in parallel such that multiple instructions are retrieved, loaded, and/or executed together. Thus, such embodiments can produce specifically-configured machines performing the steps or operations specified in the block diagrams and flow diagrams. Accordingly, the block diagrams and flow diagrams support various combinations of embodiments for performing the specified instructions, operations, or steps.

[0029] Applications may perform computations of large amounts of data. As such types of computations increase, the demand for memory may also increase. Memory expansion technologies may help alleviate this problem by providing tiered memory subsystems that help increase memory capac-

ity at a lower cost. The memory subsystem may include, for example, a memory device that adheres to a Compute Express Link (CXL) protocol. The memory device may include a volatile memory (e.g., a dynamic random access memory (DRAM)) and a non-volatile memory (NVM).

[0030] An application running on the host device may read and write data to the memory device via load and store commands, treating the memory device as an expansion of the main memory that is attached to the processor. Latencies are generally involved in accessing the memory device. The latencies involved may differ depending on the storage medium storing the data that is to be retrieved. For example, the memory device may have both a volatile storage medium (e.g., dynamic random access memory (DRAM)) and a non-volatile storage medium (e.g., NAND flash memory). The latencies of the volatile storage medium may be lower than the latencies of the non-volatile storage medium. It may be desirable to use the volatile storage medium as cache memory where a block, chunk, or page of data (collectively referred to as a “page”) that is anticipated to be accessed soon, may be moved (e.g., prefetched) to the cache memory to minimize a cache miss that will result in access of the non-volatile storage medium.

[0031] A prefetching algorithm may be used to identify the data that is to be prefetched to the cache memory. The prefetching algorithm may predict access or temperature of the page in making the prefetching decision. For example, preference may be given to more accessed or “hot” pages than less accessed or “cold” pages, for determining the pages that are to be prefetched into the cache memory. A heat map of the application may be generated for assessing page temperature. The heat map, however, is typically generated offline. Applying an offline-generated heat map at run-time may pose a challenge due to the overhead introduced in translating the virtual addresses included in the heat map, to physical addresses.

[0032] In general terms, embodiments of the present disclosure are directed to systems and methods for using a two-level clustering model for adaptively grouping physical memory addresses accessed by a host. The two-level clustering model may include a cluster at a first level, and one or more active regions of the cluster at a second level. Access of memory addresses grouped into different active regions may be monitored by the memory device for making prefetching decisions.

[0033] In some embodiments, the selection of a cluster or active region to which an accessed physical memory address is to be grouped may depend on the distance (e.g., a mathematical difference) between the accessed address and the address of the center of the cluster or the center of the active region. In some embodiments, as addresses are added to the cluster, the center of the cluster may adjust.

[0034] In some embodiments, one or more counters and/or parameters are maintained for the clusters and active regions. The counters and/or parameters may be updated as physical memory addresses are added to the clusters and active regions. For example, an activity level of an active region may be updated based on accesses of memory addresses associated with the active regions (simply referred to as access of the active regions). In some embodiments, the activity level assigned to an active region determines an amount of prefetching requests that may be processed for the region.

[0035] In some embodiments, an evaluation task is executed periodically to delete active regions that have become inactive. The evaluation task may also be configured to compute a score of remaining active regions for demoting or promoting the active regions to different activity levels. The demotion or promotion of active regions may be based on a number and timing of accesses of the active region, hit or miss ratios, and/or the like.

[0036] In some embodiments, the accesses of an active region are analyzed for determining an access profile. The access profile may indicate, for example, whether the accesses are continuous or sequential, whether the accesses have a pattern, whether the accesses are random jumps to a hot region, and/or the like. Details on systems and methods for detecting patterns of memory accesses are described in further detail in “Systems and Methods for Pattern Recognition,” filed on even date herewith, the content of which is incorporated herein by reference. Details on systems and methods for detecting random jumps to hot regions within an active region are described in further detail in “Systems and Methods for Identifying Regions of a Memory Device,” filed on even date herewith, the content of which is incorporated herein by reference.

[0037] In some embodiments, a prefetching decision is made for an active region based on identification of an access profile of the active region. For example, if the access profile indicates a sequential access, a next memory address in the sequence may be prefetched from a non-volatile memory of the storage device into a cache memory of the device. The prefetching may result in retrieval of the data before an application actually transmits a request for the data, or prior to the memory device processing the request for the data.

[0038] The use of a two-level clustering model to group accessed memory addresses may be useful when the memory accesses are by multiple applications running on the host. When multiple applications are executed (e.g., concurrently) by the host, the applications may access different portions of the memory. The memory accesses, when considered as a whole, may appear random. The dividing of the memory accesses into clusters and active regions may allow analysis of an area of the memory that may be associated with one application. The analysis on a more focused or limited area may allow the identification of access profiles that may indicate sequences, patterns, or hot regions that may not have been identifiable when considering memory accesses by the multiple application as a whole.

[0039] FIG. 1 depicts a block diagram of a system for adaptive clustering according to one or more embodiments. The system includes a host computing device (referred to as the “host”) 100 coupled to a one or more endpoints such as, for example, one or more memory devices 102a-102c (collectively referenced as 102).

[0040] The host 100 includes, without limitation, a processor 105, main memory 104, and root complex (RC) interface 112. The processor 105 may include one or more central processing unit (CPU) cores 116 configured to execute computer program instructions and process data stored in a cache memory (simply referred to as “memory” or “cache”) 118. The cache 118 may be dedicated to one of the CPU cores 116 or shared by various ones of the CPU cores. It should be appreciated that although a CPU is used to describe the various embodiments, a person of skill in the

art will recognize that a GPU or other computing unit may be used in lieu or in addition to a CPU.

[0041] The cache 118 may be coupled to a memory controller 120 which in turn is coupled to the main memory 104. The main memory 104 may include, for example, a dynamic random access memory (DRAM) storing computer program instructions and/or other types of data (collectively referenced as data) similar to the memory device 102. In order for a CPU core 116 to execute instructions or retrieve data provided by the memory device 102, the corresponding data may be loaded into the cache memory 118, and the CPU core may consume the data (e.g., directly) from the cache memory. If the data to be consumed is not already in the cache, a cache miss may occur, and the memory device 102 may need to be queried to load the data. For example, if the data to be consumed is not in the cache 118, a cache miss logic may query the data from memory (e.g., main memory (e.g., DRAM) 104 or memory device 102) based on a mapped virtual or physical address.

[0042] In some embodiments, the processor 105 (e.g., an application running on the processor) generates data access requests for the memory devices 102. Multiple applications may be executed by the processor 105 at a given time. The multiple applications may generate numerous data access requests to the memory devices 102. One or more of the data access requests may include a virtual memory address of a location to write or read data. The processor 105 may invoke a memory management unit (MMU) 108 to translate the virtual address to a physical address for processing the request. The MMU 108 may include a translation table 110 that maps virtual addresses to physical addresses. The request transmitted to the memory device 102 for fulfilling the data access request may include the physical address corresponding to the virtual address.

[0043] In some embodiments, the host 100 exchanges signals or messages with the memory devices 102 via the RC interface 112 and interface connections 106a-106c (collectively referenced as 106). For example, the host 100 may transmit a request (e.g., a load or store request) over the RC interface 112 and interface connections 106 for reading or writing data from or to the memory devices 102. Messages from the memory devices 102 to the host 100, such as, for example, responses to the requests from the host, may be delivered over the interface connections 106 to the RC interface 112, which in turn delivers the responses to the processor 105. The memory devices 102 may further transmit signals including, for example, certain types of notifications, to the host 100 over the RC interface 112 and interface connections 106.

[0044] In some embodiments, the interface connections 106 (e.g., the connector and the protocol thereof) include a memory expansion bus such as, for example, a Compute Express Link (CXL), although embodiments are not limited thereto. For example, the interface connections 106 (e.g., the connector and the protocol thereof) may also include a general-purpose interface such as, for example, Ethernet, Universal Serial Bus (USB), and/or the like. In some embodiments, the interface connections 106 may include (or may conform to) a Cache Coherent Interconnect for Accelerators (CCIX), dual in-line memory module (DIMM) interface, Small Computer System Interface (SCSI), Fiber Channel, Serial Attached SCSI (SAS), iWARP protocol, InfiniBand protocol, 5G wireless protocol, Wi-Fi protocol, Bluetooth protocol, and/or the like.

[0045] The RC interface 112 may be, for example, a CXL interface configured to implement a root complex for connecting the processor 105 and main memory 104 to the memory devices 102. The RC interface 112 may include one or more ports 114a-114c to connect the one or more memory devices 102 to the RC. In some embodiments, the MMU 108 and/or translation table 110 may be integrated into the RC 112 interface for allowing the address translations to be implemented by the RC interface.

[0046] The memory device 102 may include one or more of a volatile computer-readable storage medium and/or non-volatile computer-readable storage medium. In some embodiments, one or more of the memory devices 102 include memory that is attached to a CPU or GPU, such as, for example, a CXL attached memory device (including volatile and persistent memory device), RDMA attached memory device, and/or the like, although embodiments are not limited thereto. The CXL attached memory device (simply referred to as CXL memory) may adhere to a CXL.mem protocol where the host 100 may access the memory using commands such as load and store commands. In this regard, the host 100 may act as a requester and the CXL memory may act as a subordinate.

[0047] In some embodiments, the memory devices 102 are included in a memory system that allows memory tiering to deliver an appropriate cost or performance profile. In this regard, the different types of storage media may be organized in a memory hierarchy or tier based on a characteristic of the storage media. The characteristic may be access latency. In some embodiments, the tier or level of a memory device increases as the access latency decreases.

[0048] In some embodiments, the one or more of the memory devices 102 are memory devices of the same or different type, that are aggregated into a storage pool. For example, the storage pool may include one or more CPU or GPU attached memory devices.

[0049] FIG. 2 depicts a block diagram of a memory device 200 according to one or more embodiments. The memory device 200 may be similar to the one or more memory devices 102 of FIG. 1. In some embodiments, the memory device 200 includes a storage controller 202, storage memory 204, and non-volatile memory (NVM) 206. The storage memory 204 may be high-performing memory of the memory device 200, and may include (or may be) volatile memory, for example, such as DRAM, but the present disclosure is not limited thereto, and the storage memory 204 may be any suitable kind of high-performing volatile or non-volatile memory. Although a single storage memory 204 is depicted for simplicity sake, a person of skill in the art should recognize that the memory device 200 may include other local memory for temporarily storing other data for the storage device.

[0050] In some embodiments, the storage memory 204 is used and managed as cache memory. In this regard, the storage memory (also referred to as a device cache memory) 204 may store copies of data stored in the NVM 206. For example, data that is to be accessed by an application in the host 100 in the near future may be copied from the NVM 206 to the storage memory 204 for allowing the data to be retrieved from the storage memory 204 instead of the NVM 206. In some embodiments, the storage memory 204 has a lower access latency than the NVM 206. Thus, in some

embodiments, accessing data from the storage memory 204 helps improve overall system performance and responsiveness.

[0051] The NVM 206 may persistently store data received, for example, from the host 100. The NVM 206 may include, for example, one or more NAND flash memory, but the present disclosure is not limited thereto, and the NVM 206 may include any suitable kind of memory for persistently storing the data according to an implementation of the memory device 200 (e.g., magnetic disks, tape, optical disks, and/or the like).

[0052] The storage controller 202 may be connected to the NVM 206 and the storage memory 204 over one or more storage interfaces 207a, 207b (collectively referenced as 207). The storage controller 202 may receive input/output (I/O) requests (e.g. load or store requests) from the host 100, and transmit commands to and from the NVM 206 and/or storage memory 204 for fulfilling the I/O requests. In this regard, the storage controller 202 may include at least one processing component embedded thereon for interfacing with the host 100, the storage memory 204, and the NVM 206. The processing component may include, for example, a digital circuit (e.g., a microcontroller, a microprocessor, a digital signal processor, or a logic device (e.g., a field programmable gate array (FPGA), an application-specific integrated circuit (ASIC), and/or the like)) capable of executing data access instructions (e.g., via firmware and/or software) to provide access to and from the data stored in the storage memory 204 or NVM 206 according to the data access instructions.

[0053] In some embodiments, the storage controller 202 includes an NVM controller 208, cache controller 210, and prefetch engine 212 (also referred to as a prefetcher). Although the NVM controller 208, cache controller 210, and prefetch engine 212 are assumed to be separate components, a person of skill in the art will recognize that one or more of the components may be combined or integrated into a single component, or further subdivided into further sub-components without departing from the spirit and scope of the inventive concept.

[0054] In some embodiments, the NVM controller 208, cache controller 210, and/or prefetch engine 212 may include, for example, a digital circuit (e.g., a microcontroller, a microprocessor, a digital signal processor, or a logic device (e.g., a field programmable gate array (FPGA), an application-specific integrated circuit (ASIC), and/or the like (collectively referenced as a processor)). The digital circuit may include a memory storing instructions (e.g., software, firmware, and/or hardware code) for being executed by the processor.

[0055] In some embodiments, the NVM controller 208 is configured to receive data access requests from the host 100. The data access requests may adhere to the CXL protocol (e.g., the CXL.io, CXL.mem, and/or CXL.cache protocol). In some embodiments, the NVM controller 208 includes a flash translation layer (FTL) 214 that receives the data access request and interfaces with the NVM 206 to read data from, and write data to, the NVM. In this regard, the FTL 214 may translate a physical address included in the data access request, to a flash block address. In doing so, the FTL 214 may engage in wear leveling to move data around the storage cells of the NVM 206 to evenly distribute the writes to the NVM 206.

[0056] In some embodiments, the prefetch engine 212 is configured to monitor memory requests from the host 100, and anticipate next requests. The data associated with the anticipated next requests may be prefetched into the device cache memory 204 for increasing the hit rate or, conversely, for minimizing the miss rate. In some embodiments, the prefetch engine 212 is configured to group the physical memory addresses of the memory requests into one or more regions (referred to as clusters) and sub-regions (referred to as active regions), using a two-level adaptive clustering model. The prefetch engine 212 may monitor the active regions for an amount of activity. The active regions may be promoted, demoted, or deleted, based on access profiles determined for the regions.

[0057] In some embodiments, data is prefetched into the device cache memory 204 based on the access profiles of the active regions. In some embodiments, the prefetch engine 212 communicates with the cache controller 210 for retrieving the data that is stored in the memory address that is to be prefetched. In some embodiments, the cache controller 210 or the FTL 214 is configured to translate a requested memory block address into a flash block address. The NMV controller 208 may retrieve the data from the flash block address, and forward the data to the cache controller 210. The cache controller 210 may select a cache address (e.g., a cache line 216) of the device cache memory 204, and store the data into the cache address.

[0058] FIG. 3 depicts a block diagram of the prefetch engine 212 according to one or more embodiments. The prefetch engine 212 may include a clustering engine 300, profiling engine 302, and evaluation engine 304. Although the clustering engine 300, profiling engine 302, and evaluation engine 304 are assumed to be separate components, a person of skill in the art will recognize that one or more of the components may be combined or integrated into a single component, or further subdivided into further sub-components without departing from the spirit and scope of the inventive concept.

[0059] In some embodiments, the clustering engine 300 is configured to identify an address of a data access request (e.g., a load request) from an application running on the host 100, and identify an existing cluster or generate a new cluster for the address according to a first level of the clustering model. The size of the cluster may be preset (e.g., 8 GB). The address may be associated or grouped into an existing cluster if the address is within a range of addresses associated with the cluster. The range of addresses may be the addresses of the NVM 206. A new cluster may be generated centered around the address if the address cannot be grouped into an existing cluster.

[0060] For a second level of the clustering model, the clustering engine 300 may identify an existing active region within an identified cluster, or create a new active region. The initial size of the active region may be preset (e.g., 8 MB). The address may be associated or grouped into an existing active region if the address is within a range of addresses associated with the active region. The range of addresses may be the addresses of the NVM 206. A new active region may be generated centered around the address if the address cannot be grouped into an existing active region. The new active region may be associated with a default activity level (e.g., the lowest activity level). The activity level may change based on tracked activity of the

active region. The size of the active region may also change based on changes to the activity level.

[0061] In some embodiments, one of the clustering engine 300, profiling engine 302, and/or evaluation engine 304 is configured to monitor or track activity of the clusters and/or active regions. One or more counters or parameter values (collectively referenced as counters) may be updated based on the monitored activity. For example, requests for memory addresses associated with the clusters and/or active regions may be monitored for updating counters relating to a number of accesses per cluster, a number of accesses per active region, last access time per active region, a number of misses per active region, a number of hits per active region, and/or the like.

[0062] In some embodiments, the profiling engine 302 is configured to generate access profiles for the active regions based on the tracked activities. The access profile may indicate, for example, whether the accesses are continuous or sequential, whether the accesses have a pattern, whether the accesses are random jumps to a hot region, and/or the like.

[0063] In some embodiments, the profiling engine 302 determines, based on the access pattern of an active region, whether one or more memory addresses are to be prefetched into the device cache memory 204, and if so, identifies the memory addresses to prefetch. Whether the memory addresses are prefetched for the active region may depend, for example, on the activity level of the active region, in addition to other cache management considerations. In some embodiments, a certain allowance or credit to generate prefetch requests is provided to the active region based on the activity level of the active region. For example, an active region with the lowest activity level (e.g., activity level 1) may be allowed or allotted a lower amount of credit than an active region with the highest activity level (e.g., activity level 3).

[0064] In some embodiments, the evaluation engine 304 is configured to evaluate the active regions on a periodic (regular or irregular) basis. In some embodiments, the time of a last access to an active region may be examined during an evaluation period. The active region may change status upon detecting a criterion. The criterion may be lack of access to the active region for a period of time. For example, the active region may be deleted (or status marked as deleted) if there has been no access to the region for three evaluation periods, although embodiments are not limited thereto.

[0065] In some embodiments, the evaluation engine 304 is configured to promote or demote an active region from one activity level to another based on a score assigned to the active region. The score may be computed based on calculations performed using the counter values. The calculations may include, for example, a calculation (p1) of cluster populations, a calculation (p2) of the population of an active region, a miss ratio (p3) in the active region, and a calculation (p4) of a last access time ratio, as follows:

$$p1 = \text{cluster counter} / \text{global counter}$$

$$p2 = \text{active region counter} / \text{cluster counter}$$

$$p3 = \text{number of misses} / (\text{number of misses} + \text{number of hits})$$

-continued

$p4$ = active region last access time/global counter

where the global counter includes a count of requests to addresses associated with a cluster, and the active region counter includes a count of requests to addresses associated with the active region.

[0066] In some embodiments, a score for an active region is computed as a weighted aggregate of $p1$, $p2$, $p3$, and $p4$ as follows:

$$\text{score} = w1p1 + w2p2 + w3p3 + w4p4$$

[0067] A higher score may indicate higher activity in the active region. A threshold minimum score associated with an activity level may need to be achieved in order for the active region to be promoted or demoted to respectively a higher or lower activity level. In some embodiments, the total number of activity levels is three, although embodiments are not limited thereto. In some embodiments, the activity level assigned to an active region determines the allowance or credit of prefetch requests that may be processed for the active region.

[0068] FIG. 4 depicts a conceptual block diagram of clusters **400a**, **400b** (collectively referenced as **400**) and active regions (ARs) **402a-402c** (collectively referenced as **402**) according to one or more embodiments. The clusters **400** and active regions **402** may be generated based on addresses identified in the data access requests (e.g., load requests) from one or more applications running on the host **100**. The clusters **400** may be of a fixed size (e.g., 8 GB), centered around a cluster center address (cluster center) **412a**, **412b** (collectively referenced as **412**) of the respective cluster **400**. The cluster center **412** may be updated as addresses are added to the cluster. In this manner, the addresses of the upper and lower boundaries of the cluster may adjust as new addresses are added, although the size of the cluster may remain fixed.

[0069] In some embodiments, the active regions **402** are initialized to a default size and centered around an active region (AR) center address (AR center) (not shown). The AR center may be updated as addresses are added to the active region **402**. The size of the active regions may also be adjusted based on activities of the active regions. For example, an active region **402** with an increased activity level may be split into two or more active regions of reduced sizes.

[0070] In the example of FIG. 4, the first cluster **400a** and the first active region **402a** may be generated based on receipt of the first address **404**. The second address **406** may belong to a range of addresses associated with the first active region **402a**. Thus, the second address **406** may be associated with the first active region **402a** upon receipt of a data access request including the second address **406**.

[0071] The third address **408** may be inside the range of addresses associated with the first cluster **400a**, but outside of the range of addresses associated with the first active region **402a**. Thus, the second active region **402b** may be created around the third address **408** upon receipt of a data access request for the third address.

[0072] The fourth address **410** may be outside of the range of addresses associated with the first cluster **400a**. Thus, the second cluster **400b** and the third active region **402c** may be created around the fourth address **410** upon receipt of a data access request for the fourth address.

[0073] FIG. 5 depicts a flow diagram of a process for adaptive clustering for data prefetching according to one or more embodiments. The process starts, and in act **502**, the clustering engine **300** identifies an address (e.g., a first memory address) of a data access request by an application running on the host **100**.

[0074] For purposes of the example of FIG. 5, it is assumed that a cluster has been identified for the address. In act **504**, the clustering engine **300** identifies an active region (e.g., a first region of a first storage medium) of the cluster based on the address. The active region may be an existing active region that encompasses the address, or a new active region generated based on the address. Whether an existing active region encompasses the address may be based on computing a mathematical difference between the address and an address (e.g., a third memory address) of the center of the active region.

[0075] In act **506**, one of the clustering engine **300**, profiling engine **302**, and/or evaluation engine **304** tracks activity of the active region. In some embodiments, the profiling engine **302** generates an access profile based on the tracking of the activity.

[0076] In the event that the access profile indicates that prefetching is appropriate for an active region, the profiling engine **302** identifies, in act **508**, a second memory address based on tracking activity of the first region.

[0077] In act **510**, the prefetch engine **212** retrieves (e.g., prefetches) data associated with the second memory address from the first storage medium (e.g., the NVM **206**) and stores in a second storage medium (e.g., cache memory **204**).

[0078] FIG. 6 depicts a flow diagram of a process for assigning an address to a cluster **400** according to one or more embodiments. The process starts, and in act **600**, the clustering engine **300** identifies an address of a data access request.

[0079] In act **602**, the clustering engine **300** determines whether there are existing clusters **400** that need to be checked for determining whether the address should be added to an existing cluster. If the answer is NO, a new cluster **400** is generated in act **604**. The cluster **400** may be of a preset size centered around the address.

[0080] If, however, there are clusters that need to be checked, the address is compared, in act **606**, against a cluster center (e.g., center address **412**), and a determination is made in act **608**, as to whether the address is within the boundaries of the cluster **400**. For example, a mathematical difference between the address and the cluster center address may be computed, and a determination may be made as to whether the difference satisfies a threshold value (e.g., is within a maximum distance from the cluster center). For example, if the cluster size is 8 GB, the maximum distance may be 4 GB from the cluster center.

[0081] If the address is within the cluster **400**, the clustering engine **300** adds the address to the cluster **400** in act **610**.

[0082] In act **612**, the clustering engine **300** updates the center address of the cluster **400**. The new center address

may be an average of the added address and previously accessed addresses that have already been associated to the cluster.

[0083] In act 614, the clustering engine 300 assigns the address to an active region as discussed in further detail with respect to FIG. 7.

[0084] FIG. 7 depicts a block diagram of a process of act 614 for assigning an address to an active region 402 according to one or more embodiments. The process starts, and in act 700, the cluster assigned to the address is identified.

[0085] In act 702, a determination is made as to whether there are existing active regions 402 to which the address could be added. If the answer is NO, a new active region 402 is created in act 704.

[0086] If there are existing active region to which the address could be added, the address is compared, in act 706, against an AR center, and a determination is made in act 708 as to whether the address is within the boundaries of the active region (referred to as an AR distance) 402. For example, a mathematical difference between the address and the AR center address may be computed, and a determination may be made as to whether the difference is within a maximum distance from the AR center.

[0087] If the address is within the active region 402, the clustering engine 300 adds the address to the active region in act 710, and updates the AR distance to be the difference between the address and the AR center address.

[0088] In act 712, one or more counters and/or parameter values associated with the active region are updated. For example, an access counter for the cluster and active region may be increased, an access time may be updated, and/or a miss or hit counter may be updated based on whether the access to the address resulted in a cache miss or hit.

[0089] FIG. 8 depicts a flow diagram of a process for evaluating active regions 402 according to one or more embodiments. The evaluation may be performed in response to the evaluation engine 304 detecting a trigger. The trigger may be, for example, detecting the threshold number (e.g., 5000) of memory accesses, detecting a certain passage of time since a last evaluation, detecting a status of the memory device, and/or the like.

[0090] In act 800, the evaluation engine 304 identifies an active region 402 that is to be evaluated.

[0091] In act 802, the evaluation engine 304 determines a time of an access (e.g., a last access) of an address associated with the active region 402.

[0092] In act 804, a determination is made as to whether a criterion for deleting the active region 402 has been satisfied. The criterion may be, for example, that no memory addresses of the active region have been accessed for a certain number (e.g., three) evaluation rounds.

[0093] If the criterion has been detected, the active region 402 is deleted in act 806. In some embodiments, if no other active regions exist in the corresponding cluster, the cluster is also deleted. In some embodiments, the deletion of the active region and/or cluster may include changing the status of the active region or cluster to indicate that it is no longer valid, or to associate the region or cluster with a “deleted” status.

[0094] If the criterion for deletion has not been detected, a score is computed in act 808. The score may be computed based on calculations performed using the counter values. For example, the score may be a weighted aggregate of the

population of the corresponding cluster, the population of the active region 402, the miss ratio in the active region, and a last access time ratio.

[0095] In act 810, a determination is made as to whether the active region 402 should be promoted or demoted from a current activity level based on the computed score. An activity level may be associated with a threshold score that is to be satisfied prior to an active region being assigned the activity level. An active region 402 having a first activity level may be promoted to a higher activity level if the score of the active region is at or above the threshold score associated with the higher activity level. An active region 402 having a first activity level may be demoted to a lower activity level if the score of the active region is at or below the threshold score associated with the lower activity level.

[0096] In act 812 the evaluation engine 304 assigns to the active region the activity level that corresponds to the computed score. In some embodiments, an active region 402 with a sufficiently high activity level (e.g., activity level 3) may be split into one or more regions. For example, an active region of a size of 32 MB that is assigned to the high activity level may be split into two active regions of size 16 MB each, where each split active region may be associated with the high activity level. The splitting of the active region may provide additional (e.g., double) credit for accessing the device cache memory 204.

[0097] In act 814, a determination is made as to whether there are more active regions 402 to evaluate. If the answer is YES, the evaluation process returns to act 800 for evaluating the other active regions.

[0098] FIG. 9 depicts a flow diagram of a prefetching process according to one or more embodiments. The process starts, and in act 900, the profiling engine 302 generates an access profile for an active region.

[0099] In act 902, the prefetch engine 212 determines whether the access profile for the active region 402 calls for a prefetching action. For example, prefetching may be warranted if the access profile indicates that the accesses are continuous or sequential, that the accesses have a pattern, that the accesses are random jumps to a hot sub-region of the active region, and/or the like.

[0100] If prefetching is warranted, the prefetch engine 212 determines, in act 904, an amount of prefetching allowance or credit for the active region 402. The amount of credit may be based on the activity level assigned to the active region 402. In some embodiments, the higher the activity level, the higher the amount of credit, up to a maximum activity level and credit.

[0101] In act 906, a determination is made as to whether prefetching credits are available for the active region 402. If the answer is NO, the prefetch engine 212 refrains from transmitting a prefetch request to the cache controller 210. If the answer is YES, the prefetch engine 212 identifies, in act 908, one or more prefetch addresses based on the access profile.

[0102] In act 910, the prefetch engine 212 transmits a prefetch command to the cache controller 210. In some embodiments, the cache controller 210 processes the prefetch command based on available capacity of the device cache memory 204. If space is available in the device cache memory the requested prefetch addresses may be prefetched from the NVM 206 to the memory 204. If space is not available in the device cache memory, data may be evicted from the memory prior to fulfilling the prefetch command.

[0103] One or more embodiments of the present disclosure may be implemented in one or more processors. The term processor may refer to one or more processors and/or one or more processing cores. The one or more processors may be hosted in a single device or distributed over multiple devices (e.g. over a cloud system). A processor may include, for example, application specific integrated circuits (ASICs), general purpose or special purpose central processing units (CPUs), digital signal processors (DSPs), graphics processing units (GPUs), and programmable logic devices such as field programmable gate arrays (FPGAs). In a processor, as used herein, each function is performed either by hardware configured, i.e., hard-wired, to perform that function, or by more general-purpose hardware, such as a CPU, configured to execute instructions stored in a non-transitory storage medium (e.g. memory). A processor may be fabricated on a single printed circuit board (PCB) or distributed over several interconnected PCBs. A processor may contain other processing circuits; for example, a processing circuit may include two processing circuits, an FPGA and a CPU, interconnected on a PCB.

[0104] It will be understood that, although the terms “first”, “second”, “third”, etc., may be used herein to describe various elements, components, regions, layers and/or sections, these elements, components, regions, layers and/or sections should not be limited by these terms. These terms are only used to distinguish one element, component, region, layer or section from another element, component, region, layer or section. Thus, a first element, component, region, layer or section discussed herein could be termed a second element, component, region, layer or section, without departing from the spirit and scope of the inventive concept.

[0105] The terminology used herein is for the purpose of describing particular embodiments only and is not intended to be limiting of the inventive concept. Also, unless explicitly stated, the embodiments described herein are not mutually exclusive. Aspects of the embodiments described herein may be combined in some implementations.

[0106] As used herein, the singular forms “a” and “an” are intended to include the plural forms as well, unless the context clearly indicates otherwise. It will be further understood that the terms “comprises” and/or “comprising”, when used in this specification, specify the presence of stated features, integers, steps, operations, elements, and/or components, but do not preclude the presence or addition of one or more other features, integers, steps, operations, elements, components, and/or groups thereof. As used herein, the term “and/or” includes any and all combinations of one or more of the associated listed items. Expressions such as “at least one of,” when preceding a list of elements, modify the entire list of elements and do not modify the individual elements of the list. Further, the use of “may” when describing embodiments of the inventive concept refers to “one or more embodiments of the present disclosure”. Also, the term “exemplary” is intended to refer to an example or illustration. As used herein, the terms “use,” “using,” and “used” may be considered synonymous with the terms “utilize,” “utilizing,” and “utilized,” respectively.

[0107] Although exemplary embodiments of systems and methods for adaptive clustering have been specifically described and illustrated herein, many modifications and variations will be apparent to those skilled in the art. Accordingly, it is to be understood that systems and methods

for adaptive clustering constructed according to principles of this disclosure may be embodied other than as specifically described herein. The disclosure is also defined in the following claims, and equivalents thereof.

[0108] The systems and methods for grouping memory addresses may contain one or more combination of features set forth in the below statements.

[0109] Statement 1. A storage device comprising: a first storage medium; a second storage medium; and a processor configured to: identify a first memory address; identify a first region of the first storage medium based on the first memory address; track activity of the first region; identify a second memory address based on tracking activity of the first region; and retrieve data associated with the second memory address from the first storage medium and store the data in the second storage medium.

[0110] Statement 2. The storage device of Statement 1, wherein the first storage medium includes non-volatile memory and the second storage medium includes volatile memory.

[0111] Statement 3. The storage device of Statement 1, wherein the first region is associated with a first range of addresses, and the processor is further configured to: identify a third memory address from the first range of addresses; compute a difference between the first memory address and the third memory address; and determine that the difference satisfies a threshold value.

[0112] Statement 4. The storage device of Statement 3, wherein the processor is further configured to: identify a second region based on the first memory address, wherein the second region is associated with a second range of addresses, wherein the second region is of a preset size.

[0113] Statement 5. The storage device of Statement 3, wherein the second region includes the first region, wherein the processor is configured to identify the first region includes the processor being configured to: identify the second region based on the first memory address; and identify the first region based on the processor being configured to identify the second region.

[0114] Statement 6. The storage device of Statement 3, wherein the processor is further configured to: update the third memory address based on the first memory address and the first range of memory addresses associated with the first region.

[0115] Statement 7. The storage device of Statement 1, wherein the processor is further configured to: assign a first value to the first region based on the activity of the first region; determine, based on the first value, a first number of accesses to the second storage medium allotted to the first region.

[0116] Statement 8. The storage device of Statement 7, wherein the processor is further configured to: identify an increase in the activity of the first region; assign a second value to the first region based on identifying the increase; and determine, based on the second value, a second number of accesses to the second storage medium allotted to the first region.

[0117] Statement 9. The storage device of Statement 7, wherein the processor is further configured to: divide the first region into a second region and a third region, wherein the second region is allotted the first number of accesses to the second storage medium, and the third region is allotted the first number of accesses to the second storage medium.

[0118] Statement 10. The storage device of Statement 7, wherein the processor is further configured to: identify a decrease in the activity of the first region; and change a status of the first region based on the decrease in the activity.

[0119] Statement 11. A method comprising: identifying a first memory address; identifying a first region of a first storage medium based on the first memory address; tracking activity of the first region; identifying a second memory address based on the tracking of the activity of the first region; and retrieving data associated with the second memory address from the first storage medium and storing the data in a second storage medium.

[0120] Statement 12. The method of Statement 11, wherein the first storage medium includes non-volatile memory and the second storage medium includes volatile memory.

[0121] Statement 13. The method of Statement 11, wherein the first region is associated with a first range of addresses, the method further comprising: identifying a third memory address from the first range of addresses; computing a difference between the first memory address and the third memory address; and determining that the difference satisfies a threshold value.

[0122] Statement 14. The method of Statement 13 further comprising: identifying a second region based on the first memory address, wherein the second region is associated with a second range of addresses, wherein the second region is of a preset size.

[0123] Statement 15. The method of Statement 13, wherein the second region includes the first region, wherein the identifying of the first region includes: identifying the second region based on the first memory address; and identifying the first region based on the identifying of the second region.

[0124] Statement 16. The method of Statement 13 further comprising: updating the third memory address based on the first memory address and the first range of memory addresses associated with the first region.

[0125] Statement 17. The method of Statement 11 further comprising: assigning a first value to the first region based on the activity of the first region; determining, based on the first value, a first number of accesses to the second storage medium allotted to the first region.

[0126] Statement 18. The method of Statement 17 further comprising: identifying an increase in the activity of the first region; assigning a second value to the first region based on identifying the increase; and determining, based on the second value, a second number of accesses to the second storage medium allotted to the first region.

[0127] Statement 19. The method of Statement 17 further comprising: dividing the first region into a second region and a third region, wherein the second region is allotted the first number of accesses to the second storage medium, and the third region is allotted the first number of accesses to the second storage medium.

[0128] Statement 20. The method of Statement 17 further comprising: identifying a decrease in the activity of the first region; and changing a status of the first region based on the decrease in the activity.

What is claimed is:

1. A storage device comprising:
 - a first storage medium;
 - a second storage medium; and

a processor configured to:

- identify a first memory address;
 - identify a first region of the first storage medium based on the first memory address;
 - track activity of the first region;
 - identify a second memory address based on tracking activity of the first region; and
 - retrieve data associated with the second memory address from the first storage medium and store the data in the second storage medium.
2. The storage device of claim 1, wherein the first storage medium includes non-volatile memory and the second storage medium includes volatile memory.
 3. The storage device of claim 1, wherein the first region is associated with a first range of addresses, and the processor is further configured to:
 - identify a third memory address from the first range of addresses;
 - compute a difference between the first memory address and the third memory address; and
 - determine that the difference satisfies a threshold value.
 4. The storage device of claim 3, wherein the processor is further configured to:
 - identify a second region based on the first memory address, wherein the second region is associated with a second range of addresses, wherein the second region is of a preset size.
 5. The storage device of claim 3, wherein the second region includes the first region, wherein the processor is configured to identify the first region includes the processor being configured to:
 - identify the second region based on the first memory address; and
 - identify the first region based on the processor being configured to identify the second region.
 6. The storage device of claim 3, wherein the processor is further configured to:
 - update the third memory address based on the first memory address and the first range of memory addresses associated with the first region.
 7. The storage device of claim 1, wherein the processor is further configured to:
 - assign a first value to the first region based on the activity of the first region;
 - determine, based on the first value, a first number of accesses to the second storage medium allotted to the first region.
 8. The storage device of claim 7, wherein the processor is further configured to:
 - identify an increase in the activity of the first region;
 - assign a second value to the first region based on identifying the increase; and
 - determine, based on the second value, a second number of accesses to the second storage medium allotted to the first region.
 9. The storage device of claim 7, wherein the processor is further configured to:
 - divide the first region into a second region and a third region, wherein the second region is allotted the first number of accesses to the second storage medium, and the third region is allotted the first number of accesses to the second storage medium.

10. The storage device of claim **7**, wherein the processor is further configured to:

identify a decrease in the activity of the first region; and
change a status of the first region based on the decrease in the activity.

11. A method comprising:

identifying a first memory address;

identifying a first region of a first storage medium based on the first memory address;

tracking activity of the first region;

identifying a second memory address based on the tracking of the activity of the first region; and

retrieving data associated with the second memory address from the first storage medium and storing the data in a second storage medium.

12. The method of claim **11**, wherein the first storage medium includes non-volatile memory and the second storage medium includes volatile memory.

13. The method of claim **11**, wherein the first region is associated with a first range of addresses, the method further comprising:

identifying a third memory address from the first range of addresses;

computing a difference between the first memory address and the third memory address; and

determining that the difference satisfies a threshold value.

14. The method of claim **13** further comprising:

identifying a second region based on the first memory address, wherein the second region is associated with a second range of addresses, wherein the second region is of a preset size.

15. The method of claim **13**, wherein the second region includes the first region, wherein the identifying of the first region includes:

identifying the second region based on the first memory address; and

identifying the first region based on the identifying of the second region.

16. The method of claim **13** further comprising:

updating the third memory address based on the first memory address and the first range of memory addresses associated with the first region.

17. The method of claim **11** further comprising:

assigning a first value to the first region based on the activity of the first region;

determining, based on the first value, a first number of accesses to the second storage medium allotted to the first region.

18. The method of claim **17** further comprising:

identifying an increase in the activity of the first region; assigning a second value to the first region based on identifying the increase; and

determining, based on the second value, a second number of accesses to the second storage medium allotted to the first region.

19. The method of claim **17** further comprising:

dividing the first region into a second region and a third region, wherein the second region is allotted the first number of accesses to the second storage medium, and the third region is allotted the first number of accesses to the second storage medium.

20. The method of claim **17** further comprising:

identifying a decrease in the activity of the first region; and

changing a status of the first region based on the decrease in the activity.

* * * * *